Практична робота № 4

Тема: Побудова REST API з використанням маршрутизатора Chi

Мета роботи

- Опанувати принципи проєктування REST API з дотриманням семантики HTTP-методів, ідемпотентності та стандартизованих кодів стану.
- Набути практичних навичок роботи з маршрутизатором `go-chi/chi/v5`: налаштуванням дерева маршрутів Radix Tree, суб-роутерів (`r.Route`) та витягуванням параметрів шляху (`chi.URLParam`).
- Засвоїти організацію коду за принципами шаруватої архітектури: відокремлення доменних моделей (`domain`), потокобезпечного сховища даних (`repository`) на базі `sync.RWMutex` та HTTP-обробників (`handler`).
- Опанувати конвеєр проміжної обробки запитів (Middleware): логування, перехоплення аварійних збоїв (Panic Recovery) та керування доступом CORS.
- Навчитися фільтрувати дані за допомогою параметрів запиту (query-параметрів) та формувати стандартизовані відповіді про помилки.

Технічне забезпечення

- Операційна система: Windows, macOS або Linux.
- Середовище розробки: Visual Studio Code (з офіційним розширенням Go) або JetBrains GoLand.
- Компілятор: Go SDK версії 1.22 або новішої.
- Термінал / CLI: bash, zsh або PowerShell.
- Система контролю версій: Git.
- Інструменти тестування API: утиліта командного рядка `curl`, HTTP-клієнт VS Code (розширення REST Client / Thunder Client) або Postman.

Короткі теоретичні відомості

1. Архітектурний стиль REST та семантика HTTP-методів

REST (**Representational State Transfer**) визначає взаємодію між клієнтом і сервером через маніпуляцію ресурсами за їхніми унікальними URI. Операції над ресурсами виражаються стандартними HTTP-методами:

- **GET** — безпечне (`safe`) та ідемпотентне отримання представлення ресурсу чи колекції;
- **POST** — створення нового підпорядкованого ресурсу (неідемпотентна операція, повертає статус `201 Created`);
- **PUT** — повна заміна або створення ресурсу за вказаним ідентифікатором (ідемпотентна операція);
- **DELETE** — видалення ресурсу (ідемпотентна операція, повертає `204 No Content` або `200 OK`).

Коди стану HTTP чітко сигналізують про результат виконання:

- `200 OK` — успішне читання або оновлення;
- `201 Created` — ресурс успішно створено;
- `204 No Content` — операцію виконано, відповідь не містить тіла;
- `400 Bad Request` — помилка валідації вхідних даних клієнта;
- `404 Not Found` — запитаний ресурс не існує;
- `500 Internal Server Error` — непередбачена внутрішня помилка сервера.

---

2. Маршрутизатор `go-chi/chi` та параметризація шляхів

Пакет `go-chi/chi/v5` є легковажним маршрутизатором, сумісним зі стандартним інтерфейсом `http.Handler`. Він використовує структуру даних Radix Tree для ефективного зіставлення URL-маршрутів.

Суб-роутери дозволяють групувати ендпоінти зі спільним префіксом шляху та набором Middleware:

```
r := chi.NewRouter()

// Монтування суб-роутера з версіонуванням
r.Route("/api/v1/devices", func(r chi.Router) { ①
    r.Get("/", listDevicesHandler)
    r.Post("/", createDeviceHandler)
```

```
7
8       // Вкладений маршрут з параметром шляху
9       r.Route("/{id}", func(r chi.Router) { ②
10          r.Get("/", getDeviceHandler)
11          r.Put("/", updateDeviceHandler)
12          r.Delete("/", deleteDeviceHandler)
13      })
14  })
15
16  // Витягування параметра шляху всередині обробника
17  func getDeviceHandler(w http.ResponseWriter, r *http.Request) {
18      deviceID := chi.URLParam(r, "id") ③
19      _ = deviceID
20  }
```

① Створення кореневого простору імен ресурсів через `r.Route`.

② Оголошення шаблону маршруту з динамічним параметром `{id}`.

③ Отримання значення параметра шляху за іменем через `chi.URLParam`.

---

3. Потокобезпечне сховище даних у пам’яті (`sync.RWMutex`)

Оскільки HTTP-сервер обробляє кожен вхідний запит в окремій горутині, прямий одночасний доступ до стандартної карти `map` призведе до паніки середовища (**concurrent map read and map write**).

Для організації безпечного доступу використовується структура з м’ютексом множинного читання `sync.RWMutex`:

```
1  type MemoryDeviceRepository struct {
2      mu      sync.RWMutex
3      storage map[string]Device
4  }
5
6  // Операція читання блокує лише запис, дозволяючи іншим горутинам читати одночасно
7  func (r *MemoryDeviceRepository) GetByID(id string) (Device, bool) {
8      r.mu.RLock() ①
9      defer r.mu.RUnlock()
10     device, exists := r.storage[id]
11     return device, exists
12 }
13
14 // Операція модифікації блокує як читання, так і запис для ексклюзивного доступу
15 func (r *MemoryDeviceRepository) Save(device Device) {
16     r.mu.Lock() ②
17     defer r.mu.Unlock()
```

```
    r.storage[device.ID] = device
}
```

① Блокування розділеного читання (`RLock`): множинні клієнти можуть одночасно читати інформацію.

② Блокування ексклюзивного запису (`Lock`): гарантує неподільність модифікації карти пам'яті.

---

4. Концепція конвеєра Middleware у Go

Middleware — це функція-обгортка, яка приймає `http.Handler` і повертає новий `http.Handler`, виконуючи додаткову логіку до або після основного обробника:

```
func LoggingMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now() ①

        // Передача керування наступному обробнику в конвеєрі
        next.ServeHTTP(w, r) ②

        // Логування після завершення обробки
        duration := time.Since(start) ③
        log.Printf("[%s] %s - %v", r.Method, r.URL.Path, duration)
    })
}
```

① Замір часу надходження вхідного HTTP-запиту.

② Виклик наступного ланки у ланцюжку обробки (хендлера або наступного middleware).

③ Фіксація загальної тривалості запиту після повернення керування.

Постановка задачі

У сучасних диспетчерських центрах та промислових IoT-системах виникає необхідність ведення централізованого обліку всіх фізичних та віртуальних вузлів мережі: датчиків температури, мережевих шлюзів, реле керування та серверів. Перед початком передачі телеметрії або команд диспетчерська служба повинна мати можливість зареєструвати пристрій, перевірити його мережевий статус, змінити конфігурацію або вивести застарілий пристрій з експлуатації.

Необхідно розробити повноцінний вебсервіс **реєстру пристроїв (Device Registry API)** на базі маршрутизатора Chi. Сервіс повинен надавати REST-інтерфейс для керування життєвим циклом пристроїв, зберігати дані у потокобезпечному in-memory сховищі, підтримувати конвеєрну обробку Middleware та повертати передбачувані відповіді у форматі JSON.

Клієнтами для цього API можуть бути вебпанелі моніторингу (через CORS), термінальні скрипти автоматизації (`curl`) та мобільні додатки чергових операторів.

Завдання

Практична робота складається з трьох послідовних рівнів. Кожен наступний рівень базується на попередньому та розширює функціональність розроблюваного рішення.

Рівень 1. Базовий: Структура проєкту, сховище RWMutex та читання/створення ресурсів

1. **Ініціалізація проєкту та структури каталогу:**
   - Створити робочу директорію та ініціалізувати Go-модуль.
   - Встановити маршрутизатор Chi:

     ```
     go get github.com/go-chi/chi/v5
     ```

   - Організувати структуру пакетів:
     - `internal/domain` — моделі даних;
     - `internal/repository` — шар збереження стану;
     - `internal/handler` — HTTP-обробники запитів.
2. **Моделювання сутності Device:**
   - Описати структуру `Device` з тегами JSON (`json:"…"`):
     - `id` (`string`) — унікальний ідентифікатор пристрою;
     - `name` (`string`) — назва пристрою;
     - `ip_address` (`string`) — мережева IP-адреса;
     - `type` (`string`) — тип вузла (наприклад: `sensor`, `gateway`, `controller`);
     - `status` (`string`) — поточний стан (`online`, `offline`);
     - `updated_at` (`time.Time`) — мітка часу останньої модифікації.
3. **Реалізація потокобезпечного In-Memory репозиторію:**
   - Оголосити інтерфейс сховища пристроїв (`DeviceRepository`).
   - Створити структуру пам'яті на базі карти `map[string]domain.Device` із захистом доступу через `sync.RWMutex`.
   - Реалізувати методи:
     - `GetAll() []domain.Device` — читання із застосуванням `RLock()`;
     - `GetByID(id string) (domain.Device, bool)` — пошук пристрою із застосуванням

RLock();

- `Create(device domain.Device) error` — додавання пристрою із застосуванням `Lock()`.

4. **Налаштування Chi та реалізація початкових хендлерів:**
   - Створити головний роутер `chi.NewRouter()`.
   - Змонтувати групу маршрутів `/api/v1/devices`.
   - Реалізувати ендпоінти:
     - `GET /api/v1/devices` — повертає повний перелік зареєстрованих пристроїв зі статус-кодом `200 OK` у форматі JSON;
     - `POST /api/v1/devices` — приймає JSON із даними нового пристрою, валідує наявність обов'язкових полів (`name`, `ip_address`, `type`), генерує унікальний ID, зберігає пристрій та повертає створений об'єкт зі статус-кодом `201 Created`;
     - `GET /api/v1/devices/{id}` — витягує `id` через `chi.URLParam(r, "id")`, знаходить пристрій у репозиторії та повертає його (`200 OK`) або віддає статус `404 Not Found`.
5. **Тестування:**
   - Запустити сервер на порту `:8080`.
   - За допомогою `curl` перевірити успішне створення пристрою, отримання списку та реакцію на неіснуючий ідентифікатор.

Рівень 2. Ключовий: Повний REST CRUD та Middleware логування

1. **Збереження функціональності Рівня 1.**
2. **Розширення репозиторію операціями модифікації та видалення:**
   - Додати метод оновлення: `Update(id string, device domain.Device) bool` (з блокуванням на запис `Lock()`).
   - Додати метод видалення: `Delete(id string) bool` (з видаленням ключа з карти під `Lock()`).
3. **Реалізація хендлерів оновлення та видалення:**
   - `PUT /api/v1/devices/{id}`:
     - витягує ID з URL-шляху;
     - десеріалізує оновлені дані з тіла запиту;
     - якщо пристрій знайдено — оновлює його поля, встановлює `updated_at = time.Now()` та повертає оновлений об'єкт зі статус-кодом `200 OK`;
     - якщо пристрій відсутній — повертає статус `404 Not Found`.
   - `DELETE /api/v1/devices/{id}`:
     - видаляє вказаний пристрій зі сховища;

    - у разі успіху повертає статус-код `204 No Content` без тіла відповіді;
    - якщо пристрій не знайдено — повертає статус `404 Not Found`.
4. **Підключення конвеєра Middleware для логування:**
    - Реалізувати власний проміжний обробник або підключити стандартний Middleware Chi (`middleware.RequestID`, `middleware.RealIP`, `middleware.Logger`).
    - Забезпечити виведення у термінал сервера структурованого журналу запитів: метод, шлях, тривалість виконання та віддалену адресу клієнта.
5. **Тестування:**
    - Виконати повний життєвий цикл ресурсу через серію запитів: Create → Read → Update → Delete → Read (переконатися у поверненні 404 після видалення).

Рівень 3. Просунутий: Фільтрація, стандартизація помилок та захисні Middleware

1. **Збереження функціональності попередніх рівнів.**
2. **Параметризація списку пристроїв (Query Filtering):**
    - Модернізувати обробник `GET /api/v1/devices`, забезпечивши читання query-параметрів з URL:
        - `status` (наприклад, `?status=online`);
        - `type` (наприклад, `?type=sensor`).
    - Передавати критерії фільтрації у репозиторій та повертати лише ті пристрої, які відповідають усім заданим параметрам. Якщо параметри відсутні — повертати повний список.
3. **Стандартизований формат відповідей про помилки:**
    - Описати єдину структуру для повернення помилок (відповідно до практик RFC 7807 Problem Details або уніфікованого формату):
        - `error` (`string`) — коротка назва помилки;
        - `message` (`string`) — детальне пояснення причини збою;
        - `status` (`int`) — HTTP-статус код;
        - `timestamp` (`time.Time`) — момент виникнення події.
    - Уніфікувати всі помилкові відповіді сервісу (невалідний JSON, відсутність обов'язкових полів у POST/PUT, ресурс не знайдено), віддаючи структурований JSON зі встановленим заголовком `Content-Type: application/json`.
4. **Інтеграція захисних Middleware:**
    - **Panic Recovery Middleware:** підключити або реалізувати обробник, який через `defer` та функцію `recover()` перехоплює непередбачені збої виконання в хендлерах, логує стек викликів та запобігає аварійній зупинці всього процесу сервера, повертаючи клієнту стандартизовану відповідь `500 Internal Server Error`.

- **CORS Middleware:** налаштувати заголовки міждоменного доступу (`Access-Control-Allow-Origin: *`, дозволені методи `GET, POST, PUT, DELETE, OPTIONS`, дозволені заголовки `Content-Type, Authorization`) для можливості виконання запитів із браузерних застосунків.

Приклад очікуваного виведення програми

Консоль запуску REST-сервера:

```
=== Мережевий реєстр пристроїв (Device Registry API) ===
[СЕРВЕР] Ініціалізація сховища даних (sync.RWMutex: active)
[СЕРВЕР] Конвеєр Middleware підключено: Logger, Recoverer, CORS
[СЕРВЕР] Маршрути Chi змонтовано на префіксі /api/v1/devices
[СЕРВЕР] HTTP-сервер запущено на порту :8080...
[HTTP] 127.0.0.1 - POST /api/v1/devices -> 201 Created (1.24ms)
[HTTP] 127.0.0.1 - GET /api/v1/devices?status=online -> 200 OK (350µs)
[HTTP] 127.0.0.1 - GET /api/v1/devices/dev-999 -> 404 Not Found (180µs)
[HTTP] 127.0.0.1 - DELETE /api/v1/devices/dev-001 -> 204 No Content (420µs)
```

Приклад запиту створення пристрою через `curl`:

```
curl -i -X POST http://localhost:8080/api/v1/devices \
  -H "Content-Type: application/json" \
  -d '{"name":"Main Thermal
Sensor","ip_address":"192.168.1.50","type":"sensor","status":"online"}'
```

```
HTTP/1.1 201 Created
Content-Type: application/json
Date: Tue, 08 Sep 2026 13:45:00 GMT
Content-Length: 178

{
  "id": "dev-001",
  "name": "Main Thermal Sensor",
  "ip_address": "192.168.1.50",
  "type": "sensor",
  "status": "online",
  "updated_at": "2026-09-08T13:45:00Z"
}
```

Приклад стандартизованої помилки 404 Not Found:

```
curl -i http://localhost:8080/api/v1/devices/unknown-device
```

```
HTTP/1.1 404 Not Found
Content-Type: application/json
Date: Tue, 08 Sep 2026 13:45:15 GMT
Content-Length: 138

{
  "error": "Resource Not Found",
  "message": "Device with ID 'unknown-device' does not exist in registry",
  "status": 404,
  "timestamp": "2026-09-08T13:45:15Z"
}
```

Контрольні питання

1. Чим відрізняється архітектурний стиль REST від віддаленого виклику процедур (RPC) за принципами адресації та набором операцій?
2. У чому полягає відмінність між безпечними (`safe`) та ідемпотентними (`idempotent`) HTTP-методами? До якої категорії належать методи `GET`, `POST`, `PUT`, `DELETE`?
3. Які переваги надає маршрутизатор Chi над стандартним `http.ServeMux` при побудові комплексних API? Що таке дерево Radix Tree?
4. Чому для in-memory репозиторію у конкурентному вебсервері обов'язково використовувати `sync.RWMutex`, а не звичайний `sync.Mutex` або незахищену карту?
5. У яких випадках повертається код стану `204 No Content`, і чому в тілі такої відповіді заборонено передавати дані?
6. Як влаштований механізм передачі контексту керування в ланцюжку Middleware (`next.ServeHTTP`)?
7. Як за допомогою вбудованої функції `recover()` організувати захист HTTP-сервера від падіння при виникненні паніки в одній із горутин-обробників?
8. Що таке CORS і чому запити від вебклієнтів до REST API блокуються браузером без відповідних HTTP-заголовків сервера?

Критерії оцінювання

Оцінювання здійснюється за шкалою повноти реалізації рівнів практичної роботи:

- **«Задовільно» (Рівень 1):** реалізовано базовий прототип, створено структуру проєкту, програма успішно компілюється та виконує базові сценарії.
- **«Добре» (Рівень 2):** виконано базовий рівень та реалізовано ключову функціональність заняття, забезпечено надійність і безпеку виконання коду.
- **«Відмінно» (Рівень 3):** повністю завершено всі три рівні завдання з дотриманням кращих інженерних практик, чистоти архітектури та оптимізації.
- **Усний захист:** обов'язкове пояснення логіки коду та впевнені відповіді на контрольні питання викладача.

Вимоги до звітності

1. Опублікувати вихідний код у Git-репозиторії.
2. Оформити файл `README.md` з описом проєкту, інструкцією запуску та обов'язковими скриншотами роботи програми (включаючи демонстрацію відсутності помилок).
3. Продемонструвати роботу програми викладачеві та захистити результати.